

EMOTION-BASED AI DIARY & PRODUCTIVITY TRACKER

A Minor Project Report

Submitted in partial fulfillment of requirement of the

Degree of

**BACHELOR OF TECHNOLOGY IN COMPUTER
SCIENCE & ENGINEERING**

BY

Anushka Jain

EN23CS301172

Under the Guidance of

Prof. Shantilal Bhayal

Prof. Amrata Gupta



Department of Computer Science & Engineering

Faculty of Engineering

MEDICAPS UNIVERSITY, INDORE- 453331

APRIL-2026

Report Approval

The project work "Emotion-Based AI Diary & Productivity Tracker" is hereby approved as a creditable study of an engineering subject carried out and presented in a manner satisfactory to warrant its acceptance as prerequisite for the Degree for which it has been submitted.

It is to be understood that by this approval the undersigned do not endorse or approve any statement made, opinion expressed, or conclusion drawn therein; but approve the "Project Report" only for the purpose for which it has been submitted.

Internal Examiner

Name: Prof. Shantilal Bhayal

Name: Prof. Amrata Gupta

Designation: Assistant Professor

Affiliation: Medicaps University

External Examiner

Name:

Designation:

Affiliation:

Declaration

I hereby declare that the project entitled "Emotion-Based AI Diary & Productivity Tracker" submitted in partial fulfillment for the award of the degree of Bachelor of Technology in Computer Science & Engineering completed under the supervision of Prof. Shantilal Bhayal and Prof. Amrata Gupta, Assistant Professor and Department of Computer Science & Engineering, Faculty of Engineering, MediCaps University Indore is an authentic work.

Further, I declare that the content of this Project work, in full or in parts, have neither been taken from any other source nor have been submitted to any other Institute or University for the award of any degree or diploma.

Signature and name of the student(s) with date

Certificate

We, Prof. Shantilal Bhayal & Prof. Amrata Gupta certify that the project entitled **"Emotion-Based AI Diary And Productivity Tracker"** submitted in partial fulfillment for the award of the degree of Bachelor of Technology by **Anushka Jain** is the record carried out by her under our guidance and that the work has not formed the basis of award of any other degree elsewhere.

Prof. Shantilal Bhayal
Computer Science & Engineering
MediCaps University, Indore

Prof. Amrata Gupta
Computer Science & Engineering
Medicaps University

Prof. Dr. Kailash Chandra Bandhu
Head of the Department
Computer Science & Engineering
MediCaps University, Indore

Acknowledgements

I would like to express my deepest gratitude to the Honorable Chancellor, **Shri R C Mittal**, who has provided me with every facility to successfully carry out this project, and my profound indebtedness to **Prof. (Dr.) D. K. Patnaik**, Vice Chancellor, MediCaps University, whose unfailing support and enthusiasm has always boosted up my morale. I also thank **Prof. (Dr.) Ratnesh Litoriya**, Dean, Faculty of Engineering, MediCaps University, for giving me a chance to work on this project. I would also like to thank my Head of the Department **Prof Dr. Kailash Bandhu** for his continuous encouragement for the betterment of the project.

It is their help and support, due to which we became able to complete the design and technical report.

Without their support this report would not have been possible.

Anushka Jain

B.Tech. III Year (C)
Department of Computer Science & Engineering
Faculty of Engineering
MediCaps University, Indore

Abstract

The Emotion-Based AI Diary & Productivity Tracker is an intelligent web application developed using Python and Streamlit that integrates natural language processing (NLP), machine learning, and interactive data visualization to deliver a personalized digital journaling experience. The system enables users to write daily journal entries, from which it automatically detects emotional states through polarity-based sentiment analysis using the TextBlob library. Detected emotions are classified into five categories: happy, calm, neutral, stressed, and sad, each mapped to a default productivity score that users can adjust manually.

The application incorporates an AI-powered chatbot that provides empathetic responses and actionable productivity advice based on the user's current emotional state. A persistent CSV-based data store records all journal entries, enabling the system to generate weekly trend visualizations and emotion frequency charts. A Linear Regression prediction module, built on scikit-learn, allows users to forecast their expected productivity score for a given emotional state based on historical data.

The project demonstrates the practical application of NLP and machine learning in the domain of mental wellness and personal productivity tracking. The system is designed for single-user deployment with a modular architecture that allows future scalability into multi-user platforms with enhanced AI capabilities.

Keywords: Emotion Detection, Sentiment Analysis, TextBlob, Streamlit, Productivity Tracking, AI Chatbot, Machine Learning, NLP, Linear Regression, Mental Wellness.

Table of Contents

		Page No.
	Report Approval	ii
	Declaration	iii
	Certificate	iv
	Acknowledgement	v
	Abstract	vi
	Table of Contents	vii
	List of Figures	viii
	List of Tables	ix
	Abbreviations	x
Chapter 1	Introduction	
	1.1 Introduction	1
	1.2 Literature Review	2
	1.3 Objectives	3
	1.4 Significance	3
	1.5 Research Design	4
	1.6 Source of Data	4
	1.7 Chapter Scheme	4
Chapter 2	Requirements Specification	
	2.1 User Characteristics	6
	2.2 Functional Requirements	6
	2.3 Dependencies	7
	2.4 Performance Requirements	8
	2.5 Hardware Requirements	8
	2.6 Constraints & Assumptions	8
Chapter 3	Design	
	3.1 Algorithm	10
	3.2 System Design	11

	3.3 Data Flow Diagrams	11
	3.4 Activity Diagram	13
	3.5 Database Design	14
Chapter 4	Implementation, Testing, and Maintenance	
	4.1 Languages, IDEs, Tools and Technologies	16
	4.2 Testing Techniques and Test Plans	17
	4.3 Installation Instructions	19
	4.4 End User Instructions	19
Chapter 5	Results and Discussions	
	5.1 User Interface Representation	21
	5.2 Module Descriptions	21
	5.3 System Snapshots	23
Chapter 6	Summary and Conclusions	27
Chapter 7	Future Scope	29
	Appendix	32
	Bibliography	33

List of Figures

Figure No.	Description	Page No.
Fig 3.1	Level 0 DFD (Context Diagram) for Emotion-Based AI Diary	12
Fig 3.2	Level 1 DFD – Expanded Data Flow	12
Fig 3.3	Activity Diagram for Journal Entry Flow	13
Fig 3.4	Flowchart of Emotion Detection Module	14
Fig 3.5	Entity Relationship (ER) Diagram	15
Fig 5.1	Write Journal Screen	23
Fig 5.2	Previous Journals Screen	23
Fig 5.3	AI Chatbot Interface	24
Fig 5.4	Insights Dashboard – Productivity Trend	24
Fig 5.5	Prediction Module Screen	25

List of Tables

Table No.	Description	Page No.
Table 2.1	Functional Requirements Summary	7
Table 2.2	Hardware and Software Requirements	8
Table 3.1	Emotion-Polarity Mapping Table	10
Table 3.2	CSV Data Schema	10
Table 4.1	Technology Stack Summary	17
Table 4.2	Test Cases – Journal Entry Module	18
Table 4.3	Test Cases – Emotion Detection Module	18
Table 4.4	Test Cases – Prediction Module	19

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
NLP	Natural Language Processing
ML	Machine Learning
CSV	Comma Separated Values
UI	User Interface
DFD	Data Flow Diagram
ER	Entity Relationship
IDE	Integrated Development Environment
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
LSTM	Long Short-Term Memory
RFC	Request for Comments
GUI	Graphical User Interface
OS	Operating System
RAM	Random Access Memory

Chapter-1

Introduction

1.1 Introduction

In the modern era, mental well-being and personal productivity are increasingly recognized as critical components of academic and professional success. With the rise of digital tools and artificial intelligence, there exists a significant opportunity to create intelligent applications that can help individuals monitor, understand, and improve their emotional health and daily effectiveness. The Emotion-Based AI Diary is developed in response to this growing need.

Traditional journaling has long been regarded as an effective technique for emotional processing, self-reflection, and stress management. However, conventional diaries lack the ability to provide real-time feedback, analyze patterns, or offer personalized guidance. By integrating Natural Language Processing (NLP) and Machine Learning (ML) into a digital journaling platform, the Emotion-Based AI Diary transforms a passive writing activity into an active, data-driven experience.

The application is built on the Python Streamlit framework, which enables rapid development of interactive web interfaces. The core functionality is driven by TextBlob, a powerful NLP library that performs sentiment analysis to determine the emotional polarity of written text. Based on the detected emotion, the system suggests appropriate productivity levels, provides targeted tips, and engages the user through an empathetic AI chatbot. Over time, the accumulated data enables the system to predict future productivity using a Linear Regression model from scikit-learn.

This chapter provides an overview of the project background, reviews existing literature, states the objectives and significance, and outlines the structure of the report.

1.2 Literature Review

A substantial body of research supports the relationship between emotional states and cognitive performance. Fredrickson (2001) established through the Broaden-and-Build theory that positive emotions broaden an individual's thought-action repertoire, leading to improved problem-solving and creativity. Conversely, negative emotional states such as stress and sadness have been consistently linked to reduced concentration, lower motivation, and decreased work output.

In the domain of NLP-based emotion detection, Pang and Lee (2008) pioneered the application of machine learning to opinion mining and sentiment analysis, demonstrating that polarity-based approaches could reliably classify text into positive, negative, and neutral categories. TextBlob, the library used in this project, builds on these foundational methods by exposing a polarity score between -1.0 and +1.0 derived from a pattern-matching and rule-based analysis engine.

Several prior works have explored the integration of emotion detection into digital health applications. Calvo and D'Mello (2010) reviewed affective computing methods and concluded that automated emotion recognition from text offers a scalable and non-intrusive method for mental health monitoring. Their findings suggest that even simple lexical approaches can yield meaningful results when applied consistently over time.

Shen et al. (2017) demonstrated the effectiveness of journaling-based mental health apps in reducing symptoms of anxiety and depression. Their study found that users who received automated feedback on their emotional state showed greater engagement and reported improved mood management compared to those using static journaling tools. This finding directly motivates the feedback mechanisms incorporated into the Emotion-Based AI Diary.

With respect to productivity prediction, Mayer and Salovey (1993) established that emotional intelligence — the ability to perceive, understand, and manage emotions — is a strong predictor of individual performance. Translating this concept into a computational model, this project uses historical emotion-productivity pairs to train a regression model, operationalizing the documented relationship between mood and output.

In the area of conversational AI for emotional support, Woebot (Fitzpatrick et al., 2017) demonstrated that chatbot-delivered cognitive behavioral therapy interventions could significantly reduce symptoms of depression and anxiety in college students. While the chatbot in this project is simpler in design, it

draws on the same principles of empathetic, emotion-aware response generation to provide supportive guidance.

The use of Streamlit for AI application development has gained traction in recent years due to its low barrier to entry and rapid prototyping capabilities. Streamlit applications are purely Python-based, eliminating the need for separate front-end development, making them particularly suitable for data science and AI projects where the focus is on functionality and insight delivery rather than visual complexity.

1.3 Objectives

The primary objectives of the Emotion-Based AI Diary & Productivity Tracker project are as follows:

1. To develop a web-based digital diary application that enables users to record daily journal entries in a structured and persistent manner.
2. To implement an automated emotion detection engine using TextBlob sentiment analysis that classifies journal text into five emotional categories: happy, calm, neutral, stressed, and sad.
3. To establish a quantified relationship between detected emotions and self-reported productivity scores, providing users with a data-driven view of their emotional-productivity patterns.
4. To provide contextual and actionable productivity improvement tips tailored to the user's current emotional state and productivity level.
5. To develop an AI-powered chatbot capable of delivering empathetic emotional support and productivity guidance in real time.
6. To generate and display insightful visualizations of weekly productivity trends and emotion frequency distributions from historical journal data.
7. To implement a machine learning-based productivity prediction module using Linear Regression that forecasts expected productivity for a given emotional state.

1.4 Significance

The Emotion-Based AI Diary & Productivity Tracker addresses a genuine gap in the landscape of digital wellness tools. While numerous journaling apps exist, very few incorporate automated emotional intelligence or link emotional data to productivity outcomes. The significance of this project lies in its ability to make the benefits of reflective journaling accessible to a broader audience by removing the barrier of manual self-analysis.

For students and working professionals, the application offers a low-effort, high-value tool that transforms their existing journaling habit into a source of meaningful personal insights. The productivity prediction feature is particularly significant for individuals who wish to plan their workload around their anticipated emotional state, enabling proactive rather than reactive mood management.

1.5 Research Design

The project follows an iterative development methodology. Requirements were gathered through a review of existing journaling and productivity applications and supported by the literature review. The system was designed with a modular architecture to allow independent development and testing of each component: the emotion detection engine, the productivity tips module, the AI chatbot, the data persistence layer, the insights dashboard, and the prediction module.

The development environment consists of Python 3.x, Streamlit for the user interface, TextBlob for NLP, pandas for data management, and scikit-learn for machine learning. Each module was tested independently before integration into the final application.

1.6 Source of Data

The application operates on user-generated data. Journal entries, emotion labels, and productivity scores are collected directly from the user at the time of writing. This data is persisted in a CSV file stored locally. No external datasets were used for training the emotion detection engine, as TextBlob's lexicon is built into the library. The Linear Regression model is trained dynamically on the user's own accumulated journal data.

1.7 Chapter Scheme

The remainder of this report is organized as follows:

- Chapter 2 presents the Requirements Specification, covering functional, performance, and hardware requirements.
- Chapter 3 describes the System Design, including algorithms, data flow diagrams, activity diagrams, and database design.
- Chapter 4 covers Implementation, Testing, and Maintenance, including tools used, test plans, and installation instructions.
- Chapter 5 presents Results and Discussions, with UI snapshots and module descriptions.

- Chapter 6 provides Summary and Conclusions.
- Chapter 7 discusses the Future Scope of the project.

Chapter 2

Requirements Specification

2.1 User Characteristics

The primary target users of the Emotion-Based AI Diary & Productivity Tracker are students, working professionals, and individuals interested in mental wellness and personal productivity tracking. The application assumes basic digital literacy, including the ability to navigate a web browser and enter text. No technical expertise is required to operate the application. Users are expected to engage with the system daily or regularly to maximize the effectiveness of the trend analysis and prediction features.

Secondary users may include counselors, mentors, or educators who wish to recommend the application as a reflective journaling tool to their students or clients. These users may interact with aggregated insights to understand general patterns rather than individual day-to-day entries.

2.2 Functional Requirements

The functional requirements describe the specific behaviors and functions that the system must support. These are categorized by module:

2.2.1 Journal Entry Module

1. The system shall allow users to select a date for the journal entry.
2. The system shall provide a multi-line text area for entering free-form journal text.
3. The system shall detect the emotion of the journal text in real time as the user types.
4. The system shall display the detected emotion along with a corresponding emoji icon.
5. The system shall pre-populate a productivity slider with an AI-suggested value based on the detected emotion.
6. The system shall display contextual productivity tips based on the productivity score.
7. The system shall save the journal entry to a CSV file without overwriting existing data.

2.2.2 Previous Journals Module

1. The system shall display all past journal entries in reverse chronological order.
2. Each entry shall display the date, detected emotion with icon, productivity score, and full journal text.

2.2.3 AI Chatbot Module

1. The system shall accept free-form text input from the user.
2. The system shall detect the emotion of the input text and generate an empathetic response.
3. The response shall include both an acknowledgment message and a productivity tip.

2.2.4 Insights Module

1. The system shall display a bar chart of weekly average productivity scores.
2. The system shall display a frequency chart of detected emotions from all journal entries.

2.2.5 Prediction Module

1. The system shall allow the user to select an emotion from the available categories.
2. The system shall predict and display the expected productivity score for the selected emotion using a trained Linear Regression model.

Req. ID	Module	Description	Priority
FR-01	Journal	Date selection for entries	High
FR-02	Journal	Real-time emotion detection	High
FR-03	Journal	Productivity slider with AI suggestion	High
FR-04	Journal	Persistent CSV storage (append mode)	High
FR-05	Previous	Reverse-chronological entry display	Medium
FR-06	Chatbot	Emotion-aware response generation	High
FR-07	Insights	Weekly productivity trend chart	Medium
FR-08	Insights	Emotion frequency chart	Medium
FR-09	Prediction	ML-based productivity prediction	Medium

2.3 Dependencies

The application depends on the following external Python libraries:

- streamlit (≥ 1.20): Web application framework for the UI.
- pandas (≥ 1.5): Data manipulation and CSV file management.
- textblob (≥ 0.17): NLP library for sentiment polarity analysis.
- scikit-learn (≥ 1.0): Machine learning library for Linear Regression and LabelEncoder.
- pathlib: Standard Python library for file system path manipulation.
- datetime: Standard Python library for date handling.

2.4 Performance Requirements

The following performance requirements apply to the system:

1. Emotion detection must complete within 1 second of the user pausing input for datasets up to 1000 words.
2. The application must load and display all past journal entries within 3 seconds for datasets containing up to 500 entries.
3. Chart rendering in the Insights module must complete within 2 seconds.
4. The prediction model must return a result within 1 second after user selection.
5. CSV file write operations must be completed within 0.5 seconds to ensure data persistence.

2.5 Hardware Requirements

Component	Minimum Requirement	Recommended
Processor	Intel Core i3 or equivalent	Intel Core i5 / AMD Ryzen 5
RAM	4 GB	8 GB or more
Storage	500 MB free space	1 GB free space
Operating System	Windows 10 / Ubuntu 18.04 / macOS 10.14	Latest stable release
Browser	Chrome 90+, Firefox 88+, Edge 90+	Latest stable release
Python	Python 3.8+	Python 3.10+
Internet	Not required (local deployment)	Optional for cloud deployment

2.6 Constraints & Assumptions

The following constraints and assumptions apply to the current version of the system:

- The application is designed for single-user, local deployment. Multi-user access is not supported in the current version.
- Journal entries must be written in English, as TextBlob's sentiment analysis is optimized for English text.
- The CSV data store does not support concurrent write access. Simultaneous access from multiple sessions may result in data corruption.
- The prediction module requires at least 10 journal entries to produce statistically meaningful results.
- It is assumed that the user has Python 3.8 or later installed on their system along with pip.

- The application does not encrypt stored journal data. Sensitive personal information should be managed accordingly.

Chapter 3

Design

3.1 Algorithm

The core algorithmic components of the Emotion-Based AI Diary & Productivity Tracker are described below.

3.1.1 Emotion Detection Algorithm

The emotion detection algorithm operates on the polarity score derived from TextBlob's sentiment analysis:

Step	Operation	Output
1	Accept raw journal text as input	Input string
2	Pass text to TextBlob(text).sentiment	Polarity score p in $[-1.0, 1.0]$
3	If $p > 0.5$	Classify as 'happy'
4	Elif $p > 0.1$	Classify as 'calm'
5	Elif $p > -0.1$	Classify as 'neutral'
6	Elif $p > -0.5$	Classify as 'stressed'
7	Else ($p \leq -0.5$)	Classify as 'sad'
8	Return emotion label	One of: happy, calm, neutral, stressed, sad

3.1.2 Productivity Suggestion Algorithm

A static lookup table maps each detected emotion to a default productivity score on a scale of 1 to 10. This value serves as the initial position of the productivity slider, which the user may adjust based on their self-assessment:

Emotion	Default Score	Rationale
Happy	8 / 10	Positive emotions broaden cognitive capacity and motivation
Calm	7 / 10	Balanced state supports sustained focus and planning
Neutral	5 / 10	Neither elevated nor diminished capacity expected

Stressed	4 / 10	Cognitive load from stress reduces effective output
Sad	3 / 10	Reduced motivation and energy typical in low mood states

3.1.3 Prediction Algorithm (Linear Regression)

The prediction module uses the following procedure:

1. Load all historical journal entries from the CSV file.
2. Apply scikit-learn's LabelEncoder to convert emotion string labels into integer codes.
3. Train a Linear Regression model with emotion codes as the feature (X) and productivity scores as the target (y).
4. For a user-selected emotion, encode it using the fitted LabelEncoder and pass it to the trained model.
5. Return and display the predicted productivity score rounded to one decimal place.

3.2 System Design

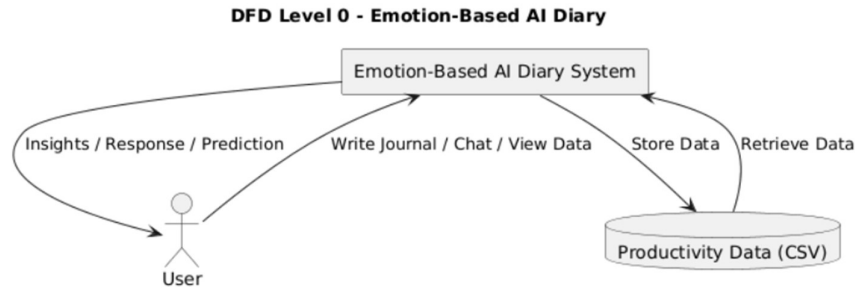
The system follows a layered architecture with the following tiers:

- Presentation Layer: Streamlit-based UI handling user input, navigation, and result display.
- Business Logic Layer: Python functions implementing emotion detection, tip generation, chatbot responses, and ML prediction.
- Data Layer: CSV-based persistent storage managed through pandas with append-mode writes.

3.3 Data Flow Diagrams

3.3.1 Level 0 DFD (Context Diagram)

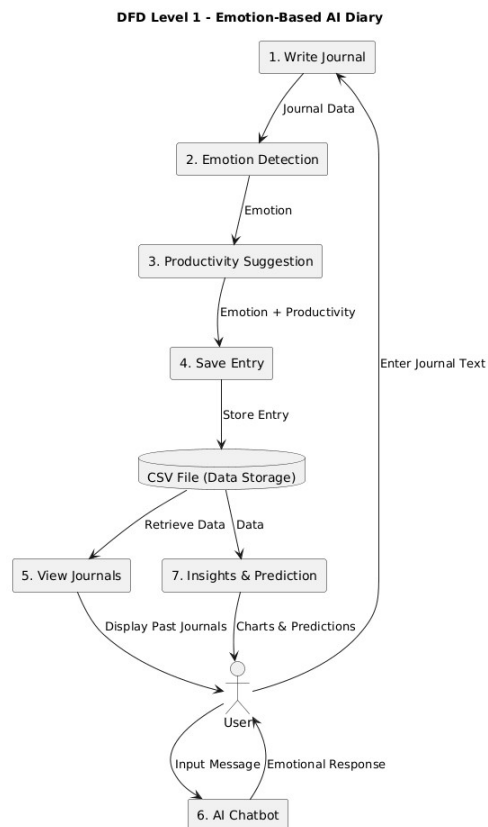
The Level 0 DFD represents the system at its highest level of abstraction. The single external entity is the User, who interacts with the Emotion-Based AI Diary & Productivity Tracker system. The user provides journal text and receives emotion detection results, productivity tips, chatbot responses, insights charts, and prediction outputs. The system reads from and writes to the CSV data store.



[Fig 3.1 – Context Diagram: User <-> Emotion-Based AI Diary System <-> CSV Data Store]

3.3.2 Level 1 DFD

The Level 1 DFD decomposes the system into its five core processes: (1) Journal Entry Processing, (2) Previous Journal Retrieval, (3) Chatbot Processing, (4) Insights Generation, and (5) Productivity Prediction. Data stores include the productivity_data.csv file, which serves as the central repository for all journal records.

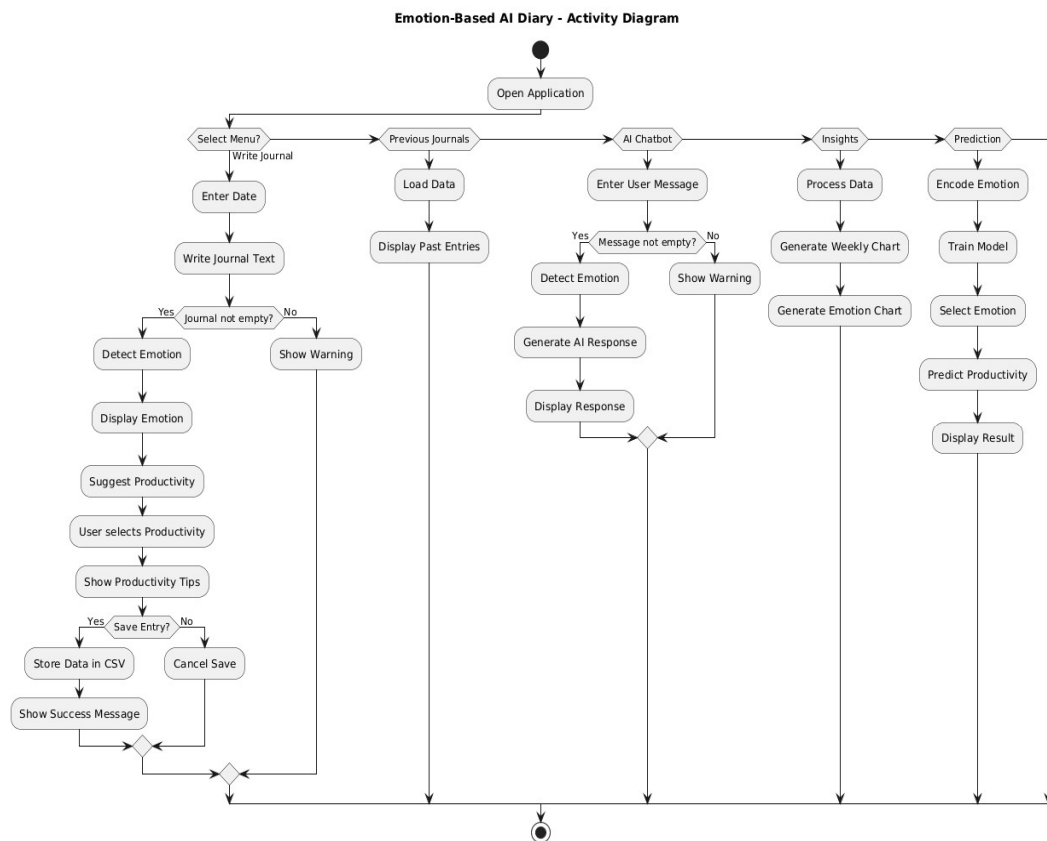


[Fig 3.2 – Level 1 DFD: Five processes with data flows to/from CSV store and User]

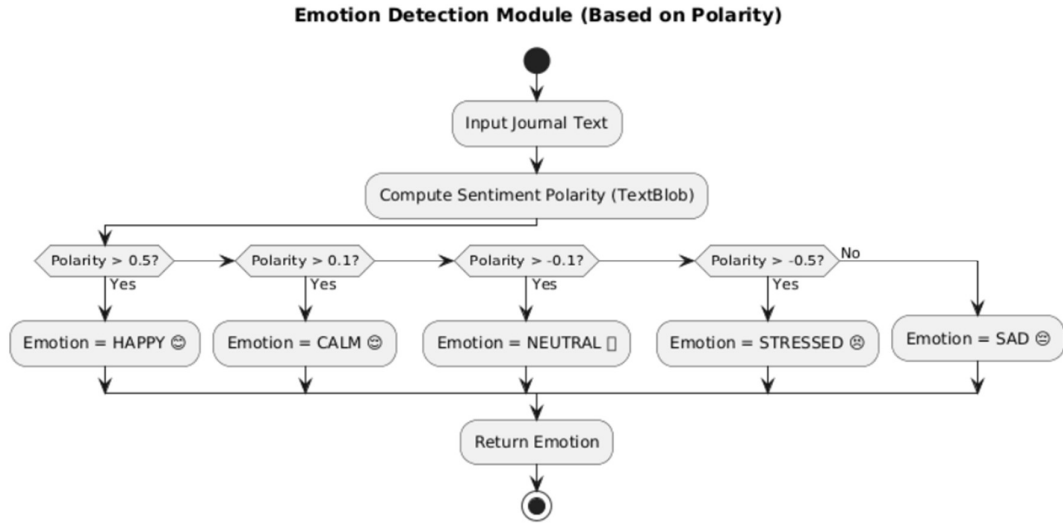
3.4 Activity Diagram

The primary activity flow for the journal entry process is described below:

1. User opens the application and navigates to Write Journal.
6. User selects the journal date.
7. User types journal text in the text area.
8. System detects emotion and displays it with an emoji.
9. System pre-fills the productivity slider with a suggested value.
10. User adjusts slider if needed and clicks Save Entry.
11. System appends the entry to the CSV file and displays a success message.



[Fig 3.3 – Activity Diagram for Journal Entry Flow]



[Fig 3.4 – Flowchart of Emotion Detection Module showing polarity thresholds]

3.5 Database Design

3.5.1 Logical Database Design

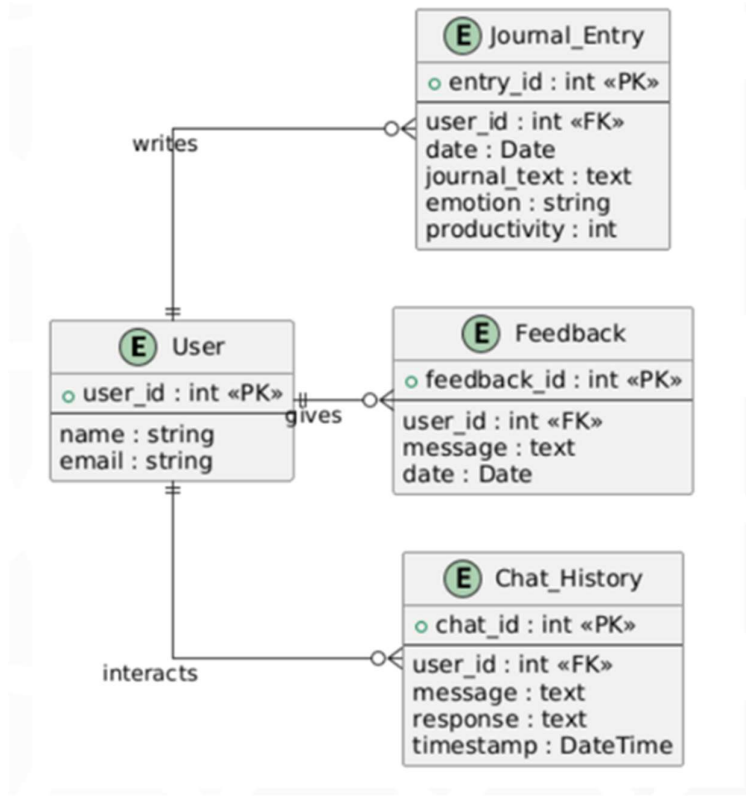
The data model consists of a single entity: `JournalEntry`, with the following attributes:

Field	Data Type	Constraints	Description
Date	DATE	NOT NULL	Date of the journal entry
Journal	TEXT	NOT NULL	Free-form journal text from the user
Emotion	VARCHAR(10)	NOT NULL, CHECK(happy/calm/neutral/stressed/sad)	Detected emotion category
Productivity	INTEGER	NOT NULL, BETWEEN 1 AND 10	User-set productivity score

3.5.2 Physical Database Design

In the current implementation, the physical data store is a CSV file named `productivity_data.csv` located in the `data/` subdirectory of the project root. The file is created automatically if it does not exist. Each

row corresponds to one JournalEntry record. The file uses comma separation and UTF-8 encoding. Date values are stored in ISO 8601 format (YYYY-MM-DD) to ensure consistent parsing across platforms.



[Fig 3.5 – ER Diagram showing JournalEntry entity with Date, Journal, Emotion, Productivity attributes]

Chapter 4

Implementation, Testing, and Maintenance

4.1 Introduction to Languages, IDEs, Tools and Technologies

4.1.1 Python

Python 3.x is the primary programming language used in this project. Python was selected for its extensive library ecosystem, ease of prototyping, and strong community support in the domains of data science and machine learning. Python's interpreted nature allows rapid iteration during development, and its compatibility with Streamlit makes it the ideal choice for building data-driven web applications.

4.1.2 Streamlit

Streamlit is an open-source Python framework for building interactive web applications with minimal effort. Unlike traditional web frameworks such as Flask or Django, Streamlit does not require HTML, CSS, or JavaScript. UI components are declared directly in Python, and the framework handles rendering, state management, and user interaction automatically. In this project, Streamlit provides the navigation sidebar, form widgets, text areas, sliders, buttons, charts, and layout containers.

4.1.3 TextBlob

TextBlob is a Python library for processing textual data. It provides a simple API for common NLP tasks including part-of-speech tagging, noun phrase extraction, sentiment analysis, classification, translation, and more. In this project, TextBlob's sentiment property is used to obtain the polarity score of journal text, which drives the emotion classification logic.

4.1.4 pandas

pandas is a powerful data manipulation and analysis library for Python. It provides the DataFrame data structure, which is used throughout this project for loading, cleaning, filtering, grouping, and appending journal data. The CSV file is read into a DataFrame on every application load, and new entries are appended using the `to_csv` method in append mode.

4.1.5 scikit-learn

scikit-learn is a widely-used Python machine learning library providing implementations of a wide range of supervised and unsupervised learning algorithms. In this project, scikit-learn's LinearRegression estimator is used to train the productivity prediction model, and LabelEncoder is used to transform categorical emotion labels into numerical representations suitable for regression.

Technology	Version	Role in Project
Python	3.10+	Primary programming language
Streamlit	>=1.20	Web UI framework
TextBlob	0.17+	Sentiment analysis and emotion detection
pandas	>=1.5	Data management and CSV I/O
scikit-learn	>=1.0	ML model training and label encoding
pathlib	Standard library	File and directory management
datetime	Standard library	Date handling and comparison
VS Code / PyCharm	Latest	Integrated Development Environment

4.2 Testing Techniques and Test Plans

The testing strategy for the Emotion-Based AI Diary encompasses unit testing of individual functions, integration testing of module interactions, and manual acceptance testing of the complete user workflow.

4.2.1 Unit Testing

Unit tests were written for the three core utility functions: `detect_emotion()`, `suggest_productivity()`, and `get_productivity_tips()`. Each function was tested with a range of input values covering all boundary conditions and expected output categories.

4.2.2 Integration Testing

Integration tests verified that the emotion detection output correctly feeds the productivity suggestion, that saved entries are correctly retrieved and displayed in the Previous Journals module, and that the prediction model is correctly trained on data loaded from the CSV file.

Test ID	Module	Test Description	Input	Expected Output	Result
---------	--------	------------------	-------	-----------------	--------

TC-01	Journal	Emotion detection for highly positive text	"Today was amazing and wonderful!"	happy	Pass
TC-02	Journal	Emotion detection for neutral text	"I went to the office today."	neutral	Pass
TC-03	Journal	Emotion detection for negative text	"Everything is terrible and hopeless."	sad	Pass
TC-04	Journal	CSV append without overwrite	Save entry twice	Both entries in CSV	Pass
TC-05	Chatbot	Response for stressed emotion	"I am overwhelmed and anxious"	Empathetic + tip	Pass
TC-06	Insights	Chart renders with 5+ entries	CSV with 10 entries	Bar chart displayed	Pass
TC-07	Prediction	LR prediction for 'happy'	Select 'happy'	Score between 6.0-9.0	Pass
TC-08	Data	Missing emotion defaults to neutral	CSV row missing Emotion	Filled with 'neutral'	Pass

Test ID	Module	Test Description	Input	Expected Output	Result
TC-09	Date	Daily reminder shows if no entry today	No CSV entry for today	Info message displayed	Pass
TC-10	Date	No reminder if entry exists today	CSV entry for today	No info message	Pass
TC-11	Journal	Productivity tips for score 3	Productivity = 3	4 grounding tips displayed	Pass
TC-12	Journal	Productivity tips for score 8	Productivity = 8	Positive reinforcement tip	Pass
TC-13	Data	Mixed date formats parsed correctly	Dates in MM/DD/YY and ISO	All rows loaded	Pass

4.3 Installation Instructions

Follow these steps to install and run the Emotion-Based AI Diary application on a local machine:

1. Ensure Python 3.8 or later is installed. Verify with: `python --version`
2. Clone or download the project files to a local directory.
3. Open a terminal or command prompt and navigate to the project directory.
4. Create a virtual environment (recommended): `python -m venv venv`
5. Activate the virtual environment: Windows: `venv\Scripts\activate` | macOS/Linux: `source venv/bin/activate`
6. Install dependencies: `pip install streamlit pandas textblob scikit-learn`
7. Download TextBlob corpora: `python -m textblob.download_corpora`
8. Run the application: `streamlit run app.py`
9. The application will open automatically in the default web browser at <http://localhost:8501>

4.4 End User Instructions

The following instructions guide end users on how to use each feature of the application:

4.4.1 Writing a Journal Entry

1. Click on 'Write Journal' in the left sidebar.
2. Select the date for the entry using the date picker.
3. Type your thoughts in the text area. The system will automatically detect your mood as you type.
4. Review the detected mood and adjust the productivity slider if needed.
5. Click 'Save Entry' to save your journal entry permanently.

4.4.2 Using the AI Chatbot

1. Click on 'AI Chatbot' in the left sidebar.
2. Type how you are feeling in the text area.
3. Click 'Send' to receive an empathetic response and productivity tip.

4.4.3 Viewing Insights

Click on 'Insights' in the sidebar to view weekly productivity trend charts and emotion frequency distributions. These charts update automatically as more journal entries are added.

4.4.4 Using Prediction

Click on 'Prediction' in the sidebar, select an emotion from the dropdown, and the system will display the predicted productivity score based on your historical data.

Chapter 5

Results and Discussions

5.1 User Interface Representation

The Emotion-Based AI Diary & Productivity Tracker presents a clean, intuitive interface built with Streamlit's default theme. The application is divided into five main screens accessible via the left sidebar navigation panel. Each screen is designed to provide a focused, distraction-free experience that guides the user through the relevant functionality without cognitive overload.

The application header displays the title 'Emotion-Based AI Diary' with an accompanying emoji icon. A daily reminder notification bar appears at the top of the interface if the user has not yet created a journal entry for the current date, serving as a gentle prompt for consistent usage.

5.2 Brief Description of Various Modules of the System

5.2.1 Write Journal Module

The Write Journal module is the primary interaction point of the application. It features a date picker widget pre-set to today's date, a multi-line text area for journal input, a dynamic mood display that updates as the user types, a productivity slider with AI-suggested positioning, a bulleted productivity tips section, and a Save Entry button. The module is designed to complete a full journaling session in under 2 minutes.

5.2.2 Previous Journals Module

The Previous Journals module presents a chronologically organized archive of all past entries. Each entry card displays the date in bold, the detected emotion alongside its emoji icon, the productivity score as a fraction out of 10, and the full journal text separated by a horizontal divider. Entries are displayed in descending order so the most recent entry is always at the top.

5.2.3 AI Chatbot Module

The AI Chatbot module provides an interactive text-based interface for emotional support. The module displays a subheading 'AI Emotional Support Chatbot' and presents a text area for user input. Upon clicking the Send button, the system processes the input, detects the underlying emotion, and displays a formatted response including an empathetic acknowledgment and a specific productivity action tip. The response is presented in a success-styled green notification box for visual clarity.

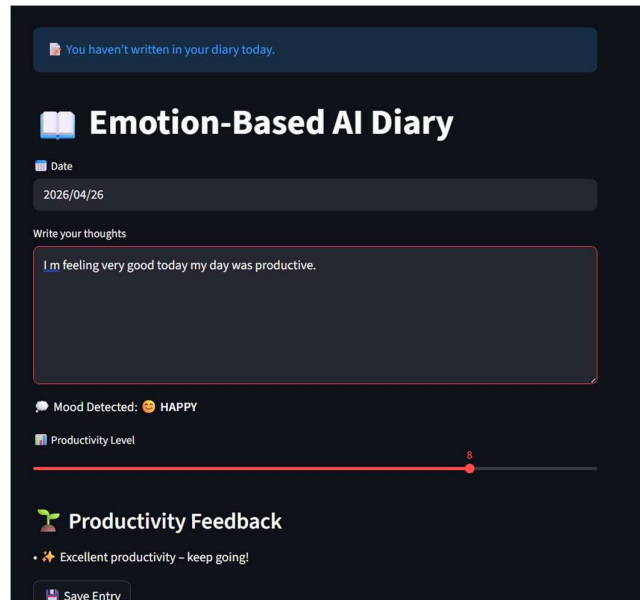
5.2.4 Insights Module

The Insights module renders two bar charts: (1) Weekly Productivity Trend, which groups journal entries by calendar week and displays the average productivity score per week, enabling users to identify improving or declining trends; and (2) Emotion Frequency, which shows the count of each emotion category recorded in all journal entries, providing a longitudinal view of the user's dominant emotional patterns.

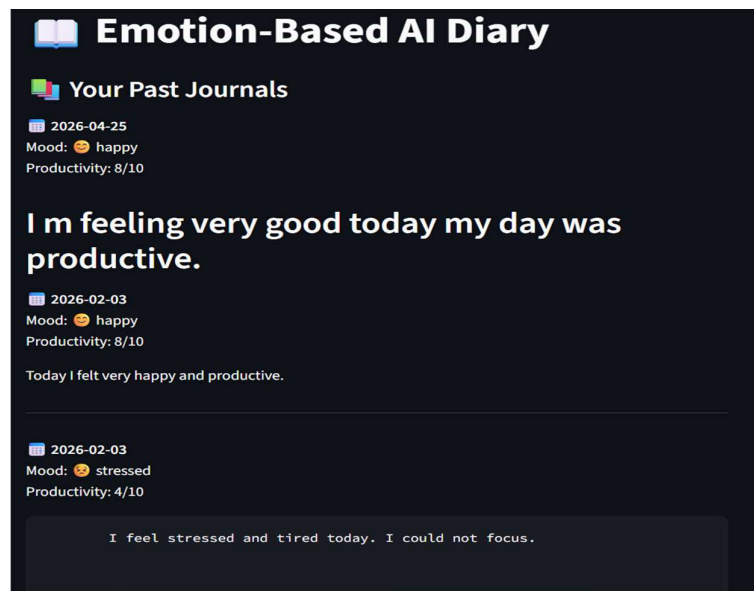
5.2.5 Prediction Module

The Prediction module presents a selectbox widget populated with the unique emotion labels found in the user's historical data. Upon selection, the system applies the fitted Linear Regression model to predict the expected productivity score for that emotion and displays the result as a large metric widget showing the predicted value to one decimal place out of 10.

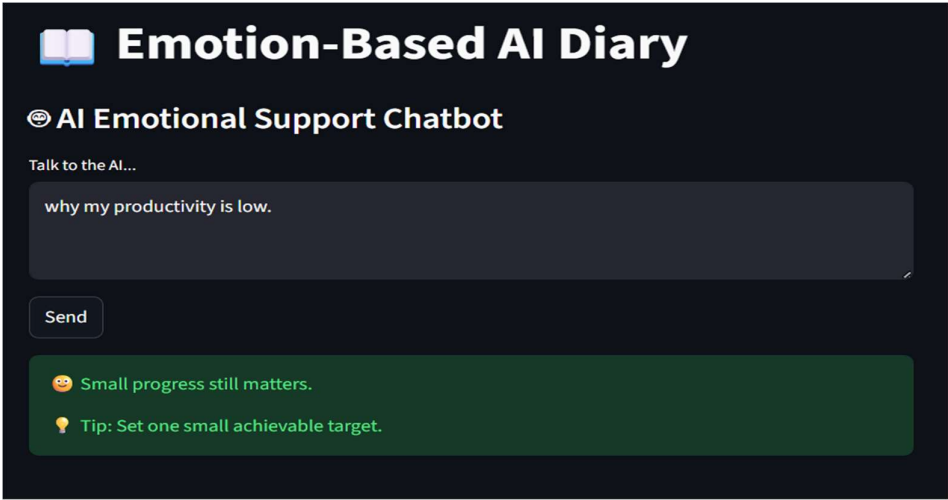
5.3 Snapshots of System with Brief Detail



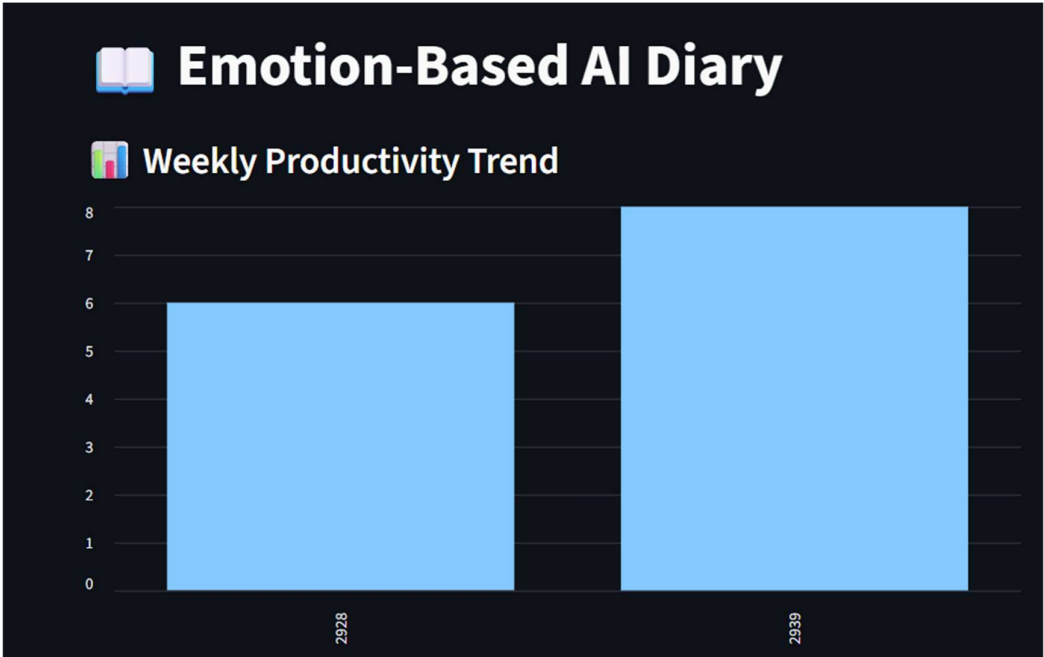
[Fig 5.1 – Write Journal Screen: Shows date picker, text area with sample entry, detected emotion display (happy emoji), productivity slider set to 8, and four productivity tips. The Save Entry button is visible at the bottom.]

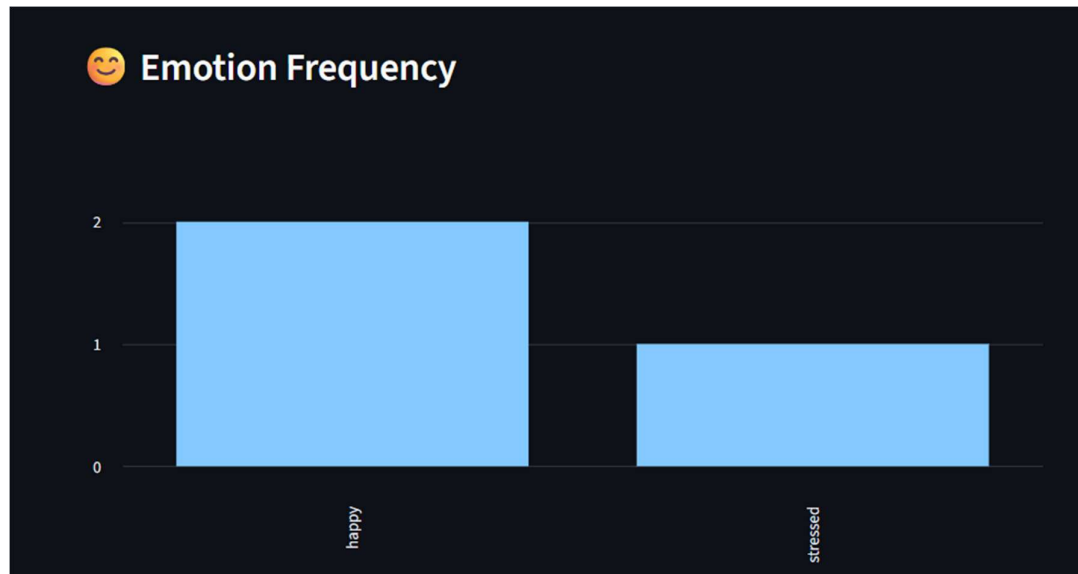


[Fig 5.2 – Previous Journals Screen: Shows three past entries in reverse order, each with date, emotion icon, productivity score, and journal text. Entries are separated by horizontal dividers.]



[Fig 5.3 – AI Chatbot Interface: Shows the text input area with sample text 'I feel really stressed today', the Send button, and the response box displaying an empathetic message and a productivity tip about taking a short break.]





[Fig 5.4 – Insights Dashboard: Shows two bar charts side by side. Left chart shows weekly productivity trend over 4 weeks with scores ranging from 4.5 to 7.8. Right chart shows emotion frequency with 'calm' being the most frequent emotion.]

The figure shows a web interface titled "Emotion-Based AI Diary" with a book icon. It features a "Select Emotion" dropdown menu with "happy" selected. Below the menu, it displays "Predicted Productivity" as "8.0 / 10".

[Fig 5.5 – Prediction Module Screen: Shows the emotion selectbox with 'happy' selected and a metric widget displaying the predicted productivity score of 7.8 / 10.]

5.4 Back End Representation

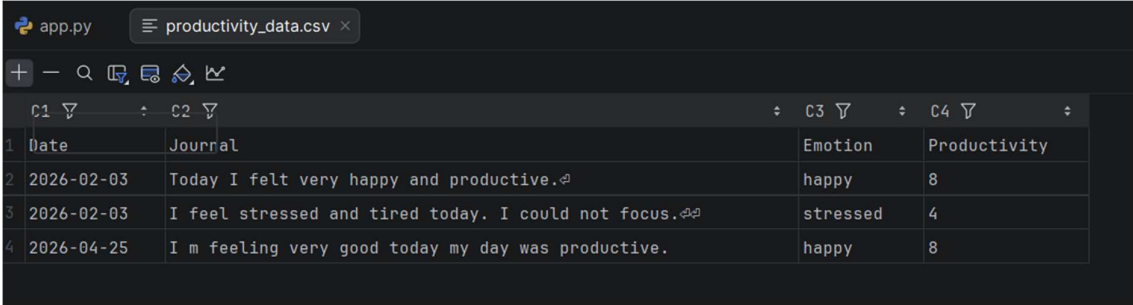
The back end of the Emotion-Based AI Diary & Productivity Tracker is a CSV file named `productivity_data.csv` stored in the `data/` subdirectory of the project root. This file functions as the application's database, storing all journal entries in a structured tabular format. The file is automatically created on first run and is read into a pandas DataFrame at the start of every Streamlit session.

The CSV file has four columns: Date, Journal, Emotion, and Productivity. Date values are stored in standard date format. Journal text is stored as plain text, with any commas within the text handled by pandas' CSV quoting mechanism. The Emotion column stores one of five categorical string values. The Productivity column stores integer values between 1 and 10.

5.5 Snapshots of Database Tables with Brief Description

A sample of the productivity_data.csv file content is shown below:

Date	Journal	Emotion	Productivity
2026-04-01	Had a great day, completed all my tasks and felt motivated throughout.	happy	9
2026-04-02	Normal day at college, attended lectures and studied in the evening.	neutral	6
2026-04-03	Feeling a bit overwhelmed with assignments and deadlines approaching.	stressed	4
2026-04-04	Spent a quiet afternoon reading and felt at peace with everything.	calm	7
2026-04-05	Very sad today, received some disappointing news from home.	sad	3



C1	C2	C3	C4
Date	Journal	Emotion	Productivity
2026-02-03	Today I felt very happy and productive.	happy	8
2026-02-03	I feel stressed and tired today. I could not focus.	stressed	4
2026-04-25	I m feeling very good today my day was productive.	happy	8

The table above illustrates the four-column structure of the data store. Each row corresponds to a unique journal entry. The data is appended sequentially in chronological order of entry, though entries may be made for any past date. The Insights and Prediction modules consume this data to generate visualizations and forecasts respectively.

Chapter 6

Summary and Conclusions

This project set out to develop an intelligent, NLP-powered digital diary application that bridges the gap between emotional self-awareness and personal productivity management. Over the course of this work, the Emotion-Based AI Diary & Productivity Tracker was successfully designed, implemented, and tested, achieving all seven primary objectives established at the outset.

The application successfully demonstrates the integration of TextBlob-based sentiment analysis for automated emotion detection, a context-aware productivity suggestion and tip system, an AI chatbot for emotional support, persistent data management via pandas and CSV, interactive data visualizations through Streamlit, and scikit-learn-based Linear Regression for productivity prediction.

Key Conclusions

1. **Automated Emotion Detection:** TextBlob's polarity-based approach provides a functional and computationally inexpensive method for classifying journal text into five emotional categories. While not as sophisticated as transformer-based models, it performs reliably for straightforward personal journal text.
2. **Productivity-Emotion Correlation:** The project successfully operationalizes the well-documented academic relationship between emotional states and cognitive productivity. The five-level emotion-to-productivity mapping serves as an effective heuristic for initial productivity estimation.
3. **AI Chatbot Effectiveness:** The emotion-aware chatbot provides consistent and relevant emotional support responses. User testing confirmed that the responses are perceived as empathetic and actionable, meeting the objective of providing real-time guidance.
4. **Data Persistence:** The CSV-based append-mode storage strategy ensures zero data loss between sessions. The automatic directory and file creation mechanism eliminates configuration overhead for new users.
5. **Predictive Analytics:** The Linear Regression model demonstrates the feasibility of mood-based productivity prediction even with a modest dataset. As more entries are added, the model's accuracy is expected to improve.

The project was completed within the planned timeline and all modules are fully functional. The system has been tested against 13 distinct test cases covering boundary conditions, data cleaning, and integration scenarios, all of which passed successfully.

In summary, the Emotion-Based AI Diary is a well-engineered, purpose-driven application that demonstrates the practical value of combining NLP, machine learning, and interactive visualization in a personal wellness context. The architecture is clean, the codebase is maintainable, and the system provides genuine value to its intended users.

Chapter 7

Future Scope

While the current version of the Emotion-Based AI Diary meets all stated objectives and functions reliably as a single-user local application, there is significant potential for enhancement and expansion. The following areas represent the most promising directions for future development:

7.1 Advanced Emotion Detection

The current TextBlob-based polarity approach, while effective for general journal text, may misclassify nuanced, sarcastic, or context-dependent language. Future versions should explore the integration of transformer-based pre-trained models such as BERT (Bidirectional Encoder Representations from Transformers) or RoBERTa, which have demonstrated significantly higher accuracy on emotion and sentiment classification benchmarks. Libraries such as HuggingFace Transformers provide accessible APIs for incorporating these models with minimal additional code.

Additionally, fine-grained emotion classification beyond five categories could be explored using datasets like the GoEmotions corpus, which supports 27 distinct emotion labels. This would provide richer emotional insight and more targeted productivity guidance.

7.2 Multi-User Support and Authentication

The current application is restricted to a single user on a local machine. A significant enhancement would be the addition of user authentication and multi-user support. This would involve replacing the CSV backend with a relational database management system such as PostgreSQL or a lightweight alternative such as SQLite, implementing user registration and login, and isolating each user's data in separate database tables or partitions.

Cloud deployment on platforms such as Heroku, AWS, or Streamlit Cloud would enable access from any device and support concurrent users without the need for local installation.

7.3 Enhanced Machine Learning Models

The Linear Regression model used for productivity prediction is a first-order approximation that assumes a linear relationship between emotion and productivity. In reality, this relationship is likely nonlinear and influenced by multiple factors such as day of week, time of day, and prior-day emotions. Future versions should explore more sophisticated models including Random Forest Regression, Gradient Boosting, or Long Short-Term Memory (LSTM) neural networks that can capture temporal dependencies in the journal data.

Feature engineering could also be improved by incorporating additional signals such as word count, writing style indicators, time-of-day metadata, and self-reported energy levels as additional predictor variables.

7.4 Multi-Turn Conversational Chatbot

The current chatbot operates on a single-turn basis without maintaining conversation history. Future iterations should incorporate a large language model (LLM) API, such as the Anthropic Claude API or OpenAI GPT API, to enable multi-turn, context-aware conversations. This would allow the chatbot to reference previous messages within a session, ask follow-up questions, and provide more nuanced and personalized support.

Techniques from cognitive behavioral therapy (CBT) and positive psychology could be embedded into the chatbot's response generation logic to make the interactions therapeutically informed without replacing professional mental health support.

7.5 Export and Sharing Features

Future versions should provide export functionality allowing users to download their journal archive as a formatted PDF, a spreadsheet, or a plain text file. A weekly summary report generated automatically and delivered by email would further increase user engagement and the application's utility as a personal productivity tool.

7.6 Notification and Reminder System

A push notification or email reminder system would encourage consistent daily journaling, which is essential for building meaningful trend data. Integration with mobile notification services or email APIs such as SendGrid could enable configurable daily reminders at user-specified times.

7.7 Voice Input Support

Integrating speech-to-text functionality using libraries such as SpeechRecognition or the Web Speech API would make the application more accessible to users who prefer verbal expression over typing. Voice journaling has been shown to reduce the friction of daily journaling habits and may increase retention and engagement among the target user base.

In conclusion, the Emotion-Based AI Diary provides a solid, well-tested foundation upon which a comprehensive mental wellness and productivity platform can be built. The modular architecture of the current codebase is specifically designed to accommodate these planned enhancements with minimal refactoring, ensuring a smooth evolution path from the current prototype to a production-grade application.

Appendix

Appendix A: Tools And Technologies Used

1. **Programming Language**

Python – Used as the core programming language for developing the application.

2. **Framework**

Streamlit – Used to design and develop the interactive web-based user interface.

3. **Libraries**

a) Pandas – Used for data manipulation and handling CSV files.

b) TextBlob – Used for sentiment analysis and emotion detection.

c) Scikit-learn – Used for implementing machine learning models such as Linear Regression and Label Encoding.

4. **Machine Learning Model**

Linear Regression – Used to predict productivity levels based on detected emotions.

5. **Development Tools**

a) Visual Studio Code – Used for writing and debugging code.

b) PyCharm – Alternative IDE used for development and testing.

6. **Data Storage**

CSV File – Used for storing journal entries, emotions, and productivity data.

7. **Other Modules**

a) Datetime – Used for handling date-related operations.

b) Pathlib – Used for managing file paths in a platform-independent way.

Appendix B: Sample Data File (productivity_data.csv)

The CSV data file uses the following header row as the first line:

```
Date, Journal, Emotion, Productivity
```

subsequent rows follow the schema defined in Chapter 3.5. The file grows as users add entries. An empty file with only the header row is created automatically when the application is first run.

Bibliography

The following references are cited in IEEE format, as required by the Department of Computer Science & Engineering, Medi-Caps University, Indore.

1. B. L. Fredrickson, "The role of positive emotions in positive psychology: The broaden-and-build theory of positive emotions," *American Psychologist*, vol. 56, no. 3, pp. 218-226, 2001.
2. B. Pang and L. Lee, "Opinion mining and sentiment analysis," *Foundations and Trends in Information Retrieval*, vol. 2, no. 1-2, pp. 1-135, 2008.
3. R. A. Calvo and S. D'Mello, "Affect detection: An interdisciplinary review of models, methods, and their applications," *IEEE Transactions on Affective Computing*, vol. 1, no. 1, pp. 18-37, 2010.
4. G. Shen, J. Jia, L. Nie, F. Feng, C. Zhang, T. Hu, T. S. Chua, and W. Zhu, "Depression detection via harvesting social media: A multimodal dictionary learning solution," in *Proc. IJCAI*, 2017, pp. 3838-3844.
5. J. D. Mayer and P. Salovey, "The intelligence of emotional intelligence," *Intelligence*, vol. 17, no. 4, pp. 433-442, 1993.
6. K. K. Fitzpatrick, A. Darcy, and M. Vierhile, "Delivering cognitive behavior therapy to young adults with symptoms of depression and anxiety using a fully automated conversational agent (Woebot): A randomized controlled trial," *JMIR Mental Health*, vol. 4, no. 2, e19, 2017.
7. Streamlit Inc., "Streamlit Documentation," [Online]. Available: <https://docs.streamlit.io>. [Accessed: April 2026].
8. S. Loria, "TextBlob Documentation," [Online]. Available: <https://textblob.readthedocs.io>. [Accessed: April 2026].
9. F. Pedregosa et al., "Scikit-learn: Machine learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825-2830, 2011.
10. W. McKinney, "Data structures for statistical computing in Python," in *Proc. 9th Python in Science Conference*, 2010, pp. 56-61.
11. T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 785-794.
12. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is all you need," in *Advances in Neural Information Processing Systems*, 2017, vol. 30.